Get unlimited access to the best of Medium for less than $1/week. Become a member

RAG I: Advanced Techniques with Chroma

Explore RAG from the basic idea into advanced techniques including Query Expansion, Cross-Encoder Re-ranking, and Query Adapters.

Sulaiman Shamasna · Follow · 28 min read · May 27, 2024

41

The content of this article is built on top of OpenAI's course: Advanced Retrieval for AI with Chroma.

Retrieval Augmented Generation (RAG) retrieves relevant documents to give context to an LLM, which makes it much better in answering queries and performing tasks. A common workflow in RAG is to take your query and embed it, then find the most similar documents (ones with similar embeddings), and that's the context. However, the problem with that is it continues to find documents that talk about similar topics of the query, but not actually contain the answer, but you can

> *Take the initial user query and rewrite it (query expansion) to pull in more directly related documents, with two key-related techniques. One, to expand the optional query into multiple queries by rewording or rewriting them in different forms. Two, to even guess or hypothesize what the answer might look like to see if we can find anything in our document collection that looks more like an answer rather than only generally talking about the topics of the query.*

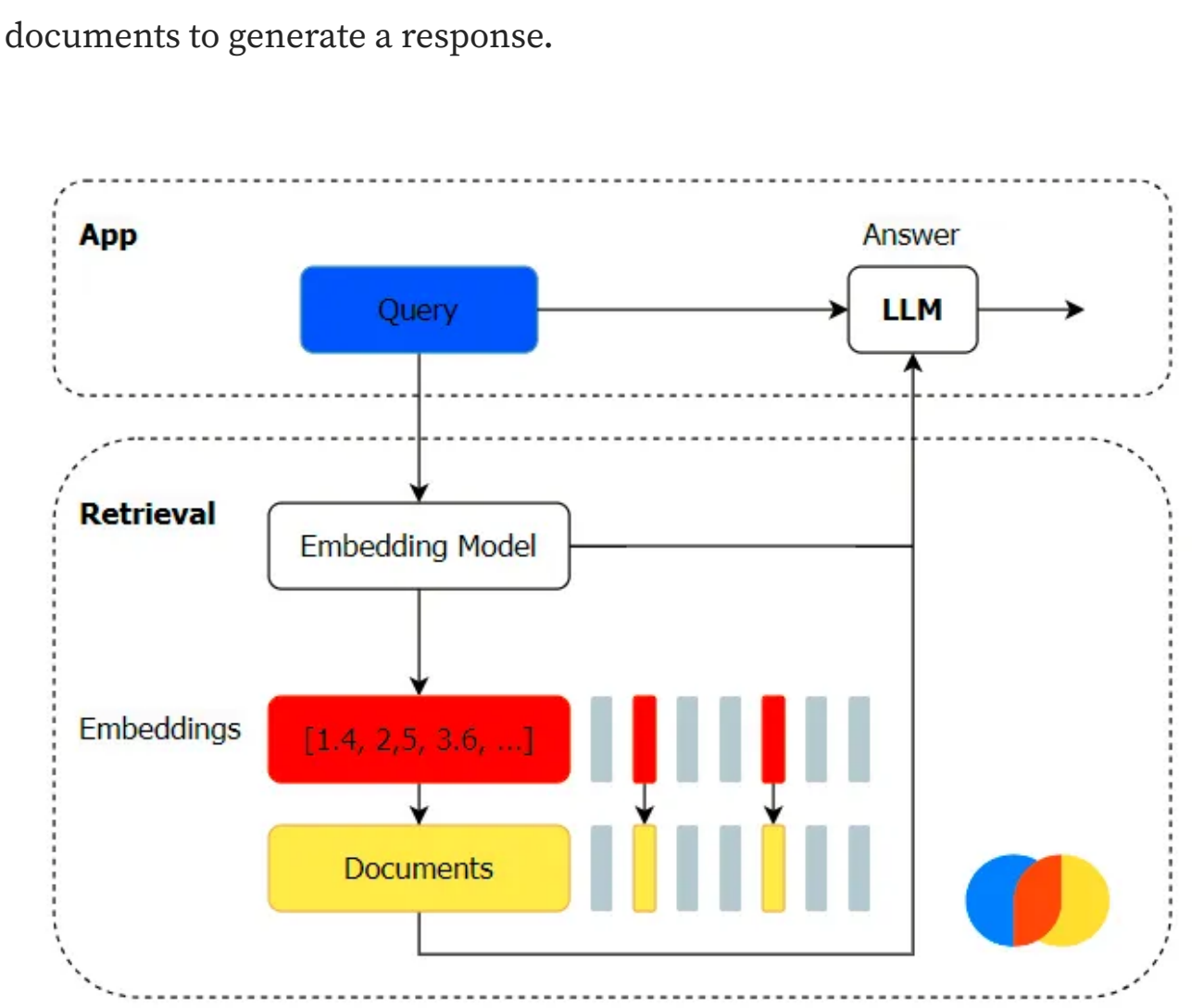
In this article, you will

- Go through a quick review of a RAG system.
- Learn about pitfalls of retrieval where vector search doesn't do well.
- Learn how to improve the results using the method of ***Query Expansion***.
- Learn how to improve the results using re-ranking with the help of ***Cross-Encoders***.
- Learn how to adapt the query embedding (***Query Adapters***) to produce more relevant results.

. . .

> ***Overview of embeddings-based retrieval***

The basic RAG process goes like that, it takes your query, runs your query through the same embedding model used to embed the documents, then the retrieval system finds the most relevant documents (the most nearest neighbor embeddings) to that query in the vector database (Chroma) in this case. Then, the relevant documents along with the query will be passed through the LLM that will synthesize information from the retrieved documents to generate a response.



Note. The following notebooks as well as the requirements file and the used documents can be found in the my ***GitHub*** repository.

The following are the steps of any RAG system.

Step 1: *Get text chunks*

```
from helper_utils import word_wrap
from pypdf import PdfReader

reader = PdfReader("microsoft_annual_report_2022.pdf")
pdf_texts = [p.extract_text().strip() for p in reader.pages]

# Filter the empty strings
pdf_texts = [text for text in pdf_texts if text]

print(word_wrap(pdf_texts[0]))
```

```
'''
OUTPUT:

1 Dear shareholders, colleagues, customers, and partners:
We are
living through a period of historic economic, societal, and
geopolitical change. The world in 2022 looks nothing like
the world in
2019. As I write this, inflation is at a 40 -year high, supply chains
are stretched, and the war in Ukraine is
ongoing. At the same time, we
are entering a technological era with the potential to power awesome
advancements
across every sector of our economy and society. As the
world's largest software company, this places us at a historic
....
'''
```

```
from langchain.text_splitter import RecursiveCharacterTextSplitter, SentenceTran

character_splitter = RecursiveCharacterTextSplitter(
    separators=["\n\n", "\n", ". ", " ", ""],
    chunk_size=1000,
    chunk_overlap=0
)
character_split_texts = character_splitter.split_text('\n\n'.join(pdf_texts))

print(word_wrap(character_split_texts[10])) # print the 10th chunk in the docume
print(f"\nTotal chunks: {len(character_split_texts)}")
```

```
'''
OUTPUT:

increased, due in large part to significant global datacenter
expansions and the growth in Xbox sales and usage. Despite
these
increases, we remain dedicated to achieving a net -zero future. We
recognize that progress won't always be linear,
and the rate at which
we can implement emissions reductions is dependent on many factors that
can fluctuate over time.
On the path to becoming water positive, we
invested in 21 water replenishment projects that are expected to
generate
over 1.3 million cubic meters of volumetric benefits in nine
water basins around the world. Progress toward our zero waste

commitment included diverting more than 15,200 metric tons of solid
waste otherwise headed to landfills and incinerators,
as well as
launching new Circular Centers to increase reuse and reduce e -waste at
our datacenters.
We contracted to protect over 17,000 acres of land
(50% more than the land we use to operate), thus achieving our

Total chunks: 347
'''
```

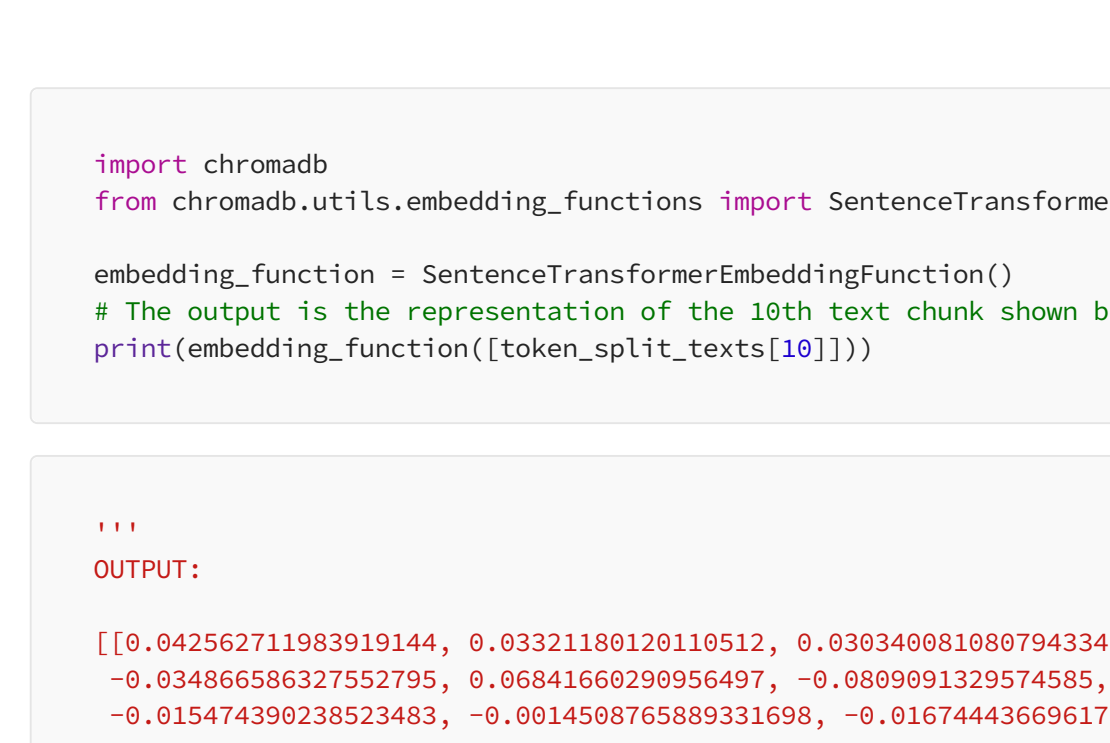
The above example splits based on character, which is not good enough, since the used embedding model (sentence transformers) has a limited context-window length (it uses *256* characters), which is the maximum window length of this embedding model. This point is so important because typically an embedding model will truncate any characters or tokens that are beyond its context window. Therefore, to make sure that all the meaning is captured in each chunk when it's going to be embedded, it's very important that the model also chunks according to the token count. Hence, the sentence transformer text splitter will be used.

```
token_splitter = SentenceTransformersTokenTextSplitter(chunk_overlap=0, tokens_p

token_split_texts = []
for text in character_split_texts:
    token_split_texts += token_splitter.split_text(text)

print(word_wrap(token_split_texts[10]))
print(f"\nTotal chunks: {len(token_split_texts)}")
```

```
'''
OUTPUT:

increased, due in large part to significant global datacenter
expansions and the growth in xbox sales and usage. despite these
increases, we remain dedicated to achieving a net - zero future. we
recognize that progress won ' t always be linear, and the rate at which
we can implement emissions reductions is dependent on many factors that
can fluctuate over time. on the path to becoming water positive, we
invested in 21 water replenishment projects that are expected to
generate over 1. 3 million cubic meters of volumetric benefits in nine
water basins around the world. progress toward our zero waste
commitment included diverting more than 15, 200 metric tons of solid
waste otherwise headed to landfills and incinerators, as well as
launching new circular centers to increase reuse and reduce e - waste
at our datacenters. we contracted to protect over 17, 000 acres of land
( 50 % more than the land we use to operate ), thus achieving our
...

Total chunks: 349
'''
```

Step 2: *Load chunks (into Chroma retriever)*

In this step, *Sentence Transformer* is gonna be used, which is an extension of the BERT transformer architecture. The BERT architecture embeds each token individually. So here we have the class of instructions token, (e.g., I like dogs), each token receives its own dense vector embedding. What a sentence transformer does is allowing you to embed an entire sentences or even small documents as shown in the figure bellow, by pooling the output of all the token embeddings to produce a single dense vector per document or in this case, per chunk.



```
import chromadb
from chromadb.utils.embedding_functions import SentenceTransformerEmbeddingFunct

embedding_function = SentenceTransformerEmbeddingFunction()
# The output is the representation of the 10th text chunk shown before (358 Dim)
print(embedding_function([token_split_texts[10]]))
```

```
'''
OUTPUT:

[[0.042562711983919144, 0.03321180120110512, 0.030340081080794334,
 -0.034866586327552795, 0.06841660290956497, -0.0809091329574585,
 -0.015474390238523483, -0.0014508765889331698, -0.01674443669617176,
 0.06770765036344528, -0.050541363656520844, -0.049195367842912674,
 0.05139990895986557, 0.09192732721567154, -0.07177840918302536,
 0.039519667625427246, -0.012833518907427788, -0.024947533383965492,
 -0.04622868448495865, -0.024357497692108154, 0.033949706703424454,
 0.02550249733030796, 0.02731710858643055, -0.004126186482608318,
 -0.03633834794163704, 0.003690890036523342, -0.027430428192019463,
 0.004796754568815231, -0.028896262869238853, -0.018870672211050987,
 0.03666623309254646, 0.025695865973830223, 0.03131285682320595,
 -0.06393438577651978, 0.05394406616687775, 0.08225351572036743,
 -0.041756849735975266, -0.006995809730142355, -0.023486023768782616,
 -0.030747992917895317, -0.002979178680106975, -0.0779094323515892,
 0.00935315527021885, 0.0031628271099179983, -0.022257082164287567,
 ...
 -0.10395055264234543, -0.023767780512571335, -0.027169033885002136]]
'''
```

```
chroma_client = chromadb.Client()
chroma_collection = chroma_client.create_collection("microsoft_annual_report_202

ids = [str(i) for i in range(len(token_split_texts))]

chroma_collection.add(ids=ids, documents=token_split_texts)
chroma_collection.count()
```

Step 3: *Connect an LLM (and build a full-fledged RAG system)*

```
query = "What was the total revenue?"

results = chroma_collection.query(query_texts=[query], n_results=5)
retrieved_documents = results['documents'][0]

for document in retrieved_documents:
    print(word_wrap(document))
    print('\n')
```

```
'''
OUTPUT:

revenue, classified by significant product and service offerings, was
as follows : ( in millions ) year ended june 30, 2022 2021 2020 server
products and cloud services $ 67, 321 $ 52, 589 $ 41, 379 office
products and cloud services 44, 862 39, 872 35, 316 windows 24, 761 22,
488 21, 510 gaming 16, 230 15, 370 11, 575 linkedin 13, 816 10, 289 8,
077 search and news advertising 11, 591 9, 267 8, 524 enterprise
services 7, 407 6, 943 6, 409 devices 6, 991 6, 791 6, 457 other 5, 291
4, 479 3, 768 total $ 198, 270 $ 168, 088 $ 143, 015 we have recast
certain previously reported amounts in the table above to conform to
the way we internally manage and monitor our business.
...
administrative 5, 900 5, 107 5, 111 operating income 83, 383 69, 916
52, 959 other income, net 333 1, 186 77 income before income taxes 83,
716 71, 102 53, 036 provision for income taxes 10, 978 9, 831 8, 755
'''
```

Now, use the previous results along with an LLM to answer the query.

```
import os
import openai
from openai import OpenAI

from dotenv import load_dotenv, find_dotenv
_ = load_dotenv(find_dotenv()) # read local .env file
openai.api_key = os.environ['OPENAI_API_KEY']

openai_client = OpenAI()
```

```
def rag(query, retrieved_documents, model="gpt-3.5-turbo"):
    information = "\n\n".join(retrieved_documents)

    # system prompt: defines how model should behave in reponse to your query
    messages = [
        {
            "role": "system",
            "content": "You are a helpful expert financial research assistant. Y
            "You will be shown the user's question, and the relevant information
        },
        {"role": "user", "content": f"Question: {query}. \n Information: {inform
    ]

    response = openai_client.chat.completions.create(
        model=model,
        messages=messages,
    )
    content = response.choices[0].message.content
    return content
```

```
output = rag(query=query, retrieved_documents=retrieved_documents)

print(word_wrap(output))
```

```
"""
OUTPUT:

The total revenue for the year ended June 30, 2022, was $198,270
million.
"""
```

The system prompt (message) is the core of the entire *RAG* loop, we're turning *GPT* from a model that remembers facts into a model that processes information, that's the system prompt.

. . .

> ***Pitfalls of Retrieval — when simple vector search fails***

This section shows where simple vector search is not really enough. Just because things are semantically close as vectors under a particular embedding model, doesn't always mean you're going to get good results right out of the box.

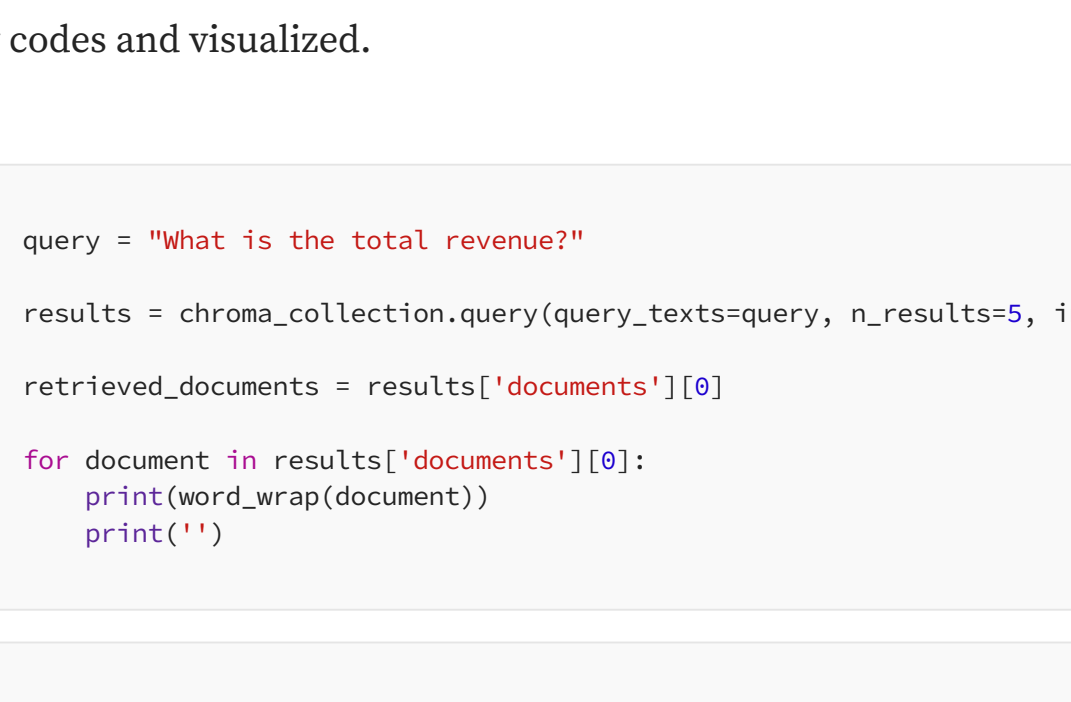
```
from helper_utils import load_chroma, word_wrap
from chromadb.utils.embedding_functions import SentenceTransformerEmbeddingFunctio

embedding_function = SentenceTransformerEmbeddingFunction()

chroma_collection = load_chroma(filename='microsoft_annual_report_2022.pdf', col
chroma_collection.count()
```

Obviously embeddings are very high dimensional, sentence transformer embeddings have *348* dimensions. However, they can be projected into 2 dimensions, which human can visualize. This can be useful for reasoning about the structure of embedding space. This is done using *UMAP* (Uniform Manifold Approximation), which is used for projecting high-dimensional data into 2 or 3 dimensions that one can visualize, which is a similar technique to *PCA* or *t-SNE*, except *UMAP* explicitly tries to preserve the structure of the data in terms of the distance between points as much as it can, unlike e.g., *PCA* which just tries to find the dominant directions and project data down.

```
import umap
import numpy as np
from tqdm import tqdm

embeddings = chroma_collection.get(include=['embeddings'])['embeddings']
umap_transform = umap.UMAP(random_state=0, transform_seed=0).fit(embeddings)

def project_embeddings(embeddings, umap_transform):
    umap_embeddings = np.empty((len(embeddings),2))
    for i, embedding in enumerate(tqdm(embeddings)):
        umap_embeddings[i] = umap_transform.transform([embedding])
    return umap_embeddings

projected_dataset_embeddings = project_embeddings(embeddings, umap_transform)
```

```
import matplotlib.pyplot as plt

plt.figure()
plt.scatter(projected_dataset_embeddings[:, 0], projected_dataset_embeddings[:,
plt.gca().set_aspect('equal', 'datalim')
plt.title('Projected Embeddings')
plt.axis('off')
```

```
(-1.3263971537351609, 9.052497419714928, 5.279644966125488, 11.5595445
63293457)
```



The representation above is the projection of the data into 2 dimensional space. Here, each dot represent a document/ context (as embeddings), and close dots to each other meaning that the context is close.

Relevancy and Distraction

In the following, terms such as relevancy and distraction will be illustrated by codes and visualized.

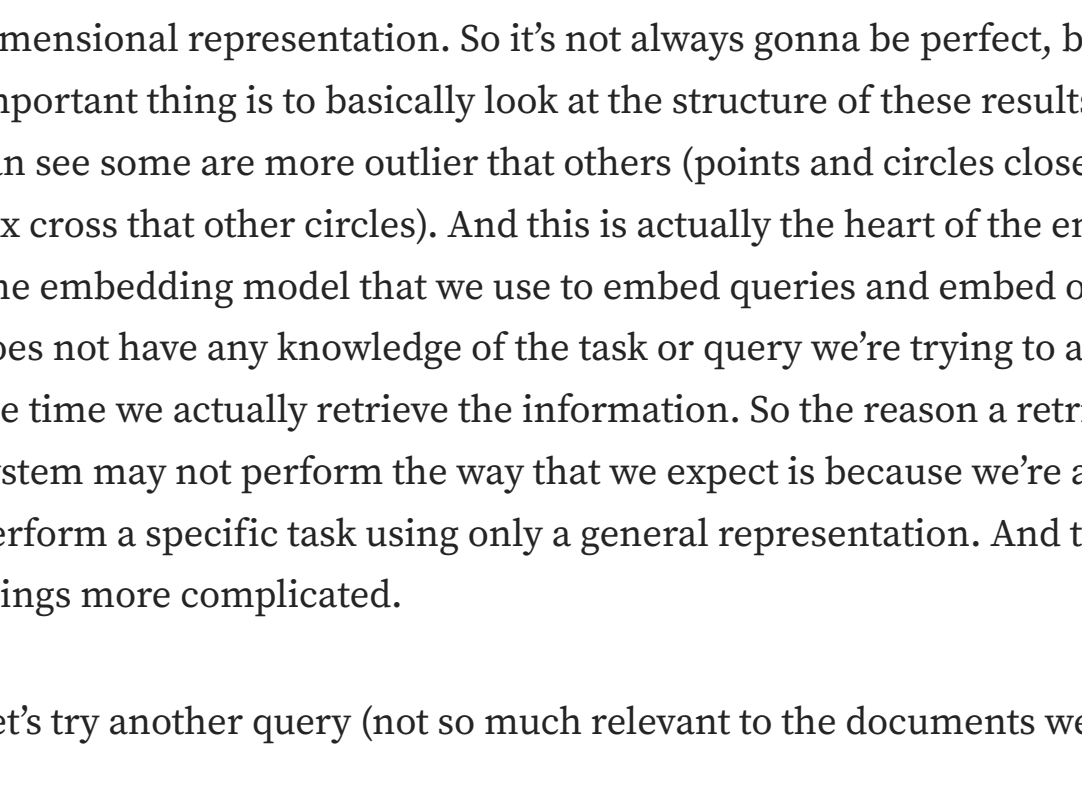
```
query = "What is the total revenue?"

results = chroma_collection.query(query_texts=query, n_results=5, include=['docu

retrieved_documents = results['documents'][0]

for document in results['documents'][0]:
    print(word_wrap(document))
    print('')
```

```
'''
OUTPUT:

revenue, classified by significant product and service offerings, was
as follows : ( in millions ) year ended june 30, 2022 2021 2020 server
products and cloud services $ 67, 321 $ 52, 589 $ 41, 379 office
products and cloud services 44, 862 39, 872 35, 316 windows 24, 761 22,
488 21, 510 gaming 16, 230 15, 370 11, 575 linkedin 13, 816 10, 289 8,
077 search and news advertising 11, 591 9, 267 8, 524 enterprise
services 7, 407 6, 943 6, 409 devices 6, 991 6, 791 6, 457 other 5, 291
4, 479 3, 768 total $ 198, 270 $ 168, 088 $ 143, 015 we have recast
...
160 other countries 98, 052 84, 135 69, 855 total $ 198, 270 $ 168, 088
$ 143, 015 ( a ) includes billings to oems and certain multinational
organizations because of the nature of these businesses and the
impracticability of determining the geographic source of the revenue.
'''
```

```
query_embedding = embedding_function([query])[0]
retrieved_embeddings = results['embeddings'][0]

projected_query_embedding = project_embeddings([query_embedding], umap_transform
projected_retrieved_embeddings = project_embeddings(retrieved_embeddings, umap_t
```

```
# Plot the projected query and retrieved documents in the embedding space
plt.figure()
plt.scatter(projected_dataset_embeddings[:, 0], projected_dataset_embeddings[:,
plt.scatter(projected_query_embedding[:, 0], projected_query_embedding[:, 1], s=
plt.scatter(projected_retrieved_embeddings[:, 0], projected_retrieved_embeddings

plt.gca().set_aspect('equal', 'datalim')
plt.title(f'{query}')
plt.axis('off')
```

```
(-1.3263971537351609, 9.052497419714928, 5.279644966125488, 11.5595445
63293457)
```



This is a visualization of the query and the retrieved embeddings. The red cross represents the query, and the green circles basically circle the data points that actually ended up being retrieved. Notice that it doesn't look in the projection like these are the actually nearest neighbors. However, remember we're trying to squash down many higher dimensions into this 2 dimensional representation. So it's not always gonna be perfect, but the important thing is to basically look at the structure of these results. And we can see some are more outlier that others (points and circles closer to the rex cross that other circles). And this is actually the heart of the entire issue. The embedding model that we use to embed queries and embed our data does not have any knowledge of the task or query we're trying to answer at the time we actually retrieve the information. So the reason a retrieval system may not perform the way that we expect is because we're asking it to perform a specific task using only a general representation. And that makes things more complicated.

Let's try another query (not so much relevant to the documents we have).

```
query = "What is the strategy around artificial intelligence (AI)?"
results = chroma_collection.query(query_texts=query, n_results=5, include=['docu

retrieved_documents = results['documents'][0]

for document in results['documents'][0]:
    print(word_wrap(document))
    print('')
```

```
'''
OUTPUT:

culture of trust can help navigate the challenges the world faces
today. organizations of all sizes have digitized business - critical
functions, redefining what they can expect from their business
applications. customers are looking to unlock value while simplifying
security and management. from infrastructure and data, to business
applications and collaboration, we provide unique, differentiated value
to customers.
...
assistance, code generation, and reasoning over our data with our new azure
openai service. digital and app innovation

• technology and research, focuses on our ai innovations and other
forward - looking research and development efforts spanning
infrastructure, services, and applications. • linkedin, focuses on our
services that transform the way customers hire, market, sell, and
```

```
learn.
'''
```

```
query_embedding = embedding_function([query])[0]
retrieved_embeddings = results['embeddings'][0]

projected_query_embedding = project_embeddings([query_embedding], umap_transform
projected_retrieved_embeddings = project_embeddings(retrieved_embeddings, umap_t
```

```
# Plot the projected query and retrieved documents in the embedding space
plt.figure()
plt.scatter(projected_dataset_embeddings[:, 0], projected_dataset_embeddings[:,
plt.scatter(projected_query_embedding[:, 0], projected_query_embedding[:, 1], s=
plt.scatter(projected_retrieved_embeddings[:, 0], projected_retrieved_embeddings

plt.gca().set_aspect('equal', 'datalim')
plt.title(f'{query}')
plt.axis('off')
```

Now with the query "What is the strategy around artificial intelligence (AI)?" If you run the model with this query using the data file given in the course, you will see that AI is mentioned in most of the documents (in the complete output shown partially above), and you will find some vaguely related to AI, you will find also information about database that is not related to AI. You can see from the 2 dimensional projection above that the point closest to the circle appears to be almost bang on where our query landed. This means it's super relevant. So one can see that, obviously, there's where the query lands in the space has geometric meaning and we're pulling in related results. But again, what's related is from the general purpose embedding model, not from the specific task we're performing. So let's take another query.

```
query = "What has been the investment in research and development?"
results = chroma_collection.query(query_texts=query, n_results=5, include=['docu

retrieved_documents = results['documents'][0]

for document in results['documents'][0]:
    print(word_wrap(document))
    print('')
```

```
'''
OUTPUT:

• operating expenses increased $ 1. 5 billion or 14 % driven by
investments in gaming, search and news advertising, and windows
marketing. operating expenses research and development ( in millions,
except percentages ) 2022 2021 percentage change research and
development $ 24, 512 $ 20, 716 18 % as a percent of revenue 12 % 12 %
0ppt research and development expenses include payroll, employee
benefits, stock - based compensation expense, and other headcount -
...
and territories with our global skills initiative, we introduced a new
initiative to support a more skills - based labor market, with greater
flexibility and accessible learning paths to develop the right skills
'''
```

```
query_embedding = embedding_function([query])[0]
retrieved_embeddings = results['embeddings'][0]

projected_query_embedding = project_embeddings([query_embedding], umap_transform
projected_retrieved_embeddings = project_embeddings(retrieved_embeddings, umap_t
```

```
# Plot the projected query and retrieved documents in the embedding space
plt.figure()
plt.scatter(projected_dataset_embeddings[:, 0], projected_dataset_embeddings[:,
plt.scatter(projected_query_embedding[:, 0], projected_query_embedding[:, 1], s=
plt.scatter(projected_retrieved_embeddings[:, 0], projected_retrieved_embeddings

plt.gca().set_aspect('equal', 'datalim')
plt.title(f'{query}')
plt.axis('off')
```

Now for the query "What has been the investment in research and development?", this is a very general query and it should be reflected in the annual statement. So as before, if you run the model on the original document provided by the course, you'll find that it starts with general ideas about investments, some of it is about research and development. However, you'll see that there's also ***Distractors*** in the results. *Distractor* is a result that is not actually relevant to the query, and it's called a distractor because if you pass this information to the *LLM* to complete your *RAG* loop, the model tends to get distracted by this information and outputs sub-optimal results. And the reason this is really important is that bad behavior from the model due to distractors is very difficult to diagnose and debug, both for the user and also for the developers and engineers building these types of systems. So it's very important to make your retrieval system robust and return relevant results and no distracting results to the model. So if we looked at the projection, you will see the results we're getting are a lot more spread out, and the way one can imagine this is imagining all their data is a cloud of points, sitting in this high-dimensional space. A query that lands inside the cloud is likely to find nearest neighbors that are kind of densely packed and close together inside the cloud. But a query that lands outside the cloud is likely to find nearest neighbors from a lot of different parts of the cloud, so they tend to be more spread out. It's a geometrical intuition. The following query would be totally irrelevent, let's check the results.

```
query = "What has Michael Jordan done for us lately?"
results = chroma_collection.query(query_texts=query, n_results=5, include=['docu

retrieved_documents = results['documents'][0]

for document in results['documents'][0]:
    print(word_wrap(document))
    print('')
```

```
'''
OUTPUT:

18 total rewards we develop dynamic, sustainable, market - driven, and
strategic programs with the goal of providing a highly differentiated
portfolio to attract, reward, and retain top talent and enable our
employees to thrive. these programs reinforce our culture and values
such as collaboration and growth mindset. managers evaluate and
recommend rewards based on, for example, how well we leverage the work
of others and contribute to the success of our colleagues. we monitor
...
institutions, enabling increased funds into local communities.
additionally, we enriched our supplier pipeline, reaching more than 90
percent of our goal to spend $ 500 million with double the number of
black and african american - owned suppliers. we
'''
```

```
query_embedding = embedding_function([query])[0]
retrieved_embeddings = results['embeddings'][0]

projected_query_embedding = project_embeddings([query_embedding], umap_transform
projected_retrieved_embeddings = project_embeddings(retrieved_embeddings, umap_t
```

```
# Plot the projected query and retrieved documents in the embedding space
plt.figure()
plt.scatter(projected_dataset_embeddings[:, 0], projected_dataset_embeddings[:,
plt.scatter(projected_query_embedding[:, 0], projected_query_embedding[:, 1], s=
plt.scatter(projected_retrieved_embeddings[:, 0], projected_retrieved_embeddings

plt.gca().set_aspect('equal', 'datalim')
plt.title(f'{query}')
plt.axis('off')
```

If you check the results, you'll definitely find noting related to Michael Jordan. And this is what we should expect. But remember, if we're using a retrieval system as a part of a *RAG* loop, you're guaranteed to return the nearest neighbors. In this case the context window is gonna be made up entirely of distractors, which as mentioned earlier, can be very difficult to understand and debug from the application user's perspective and and from the application developer's perspective. So we need a way to deal with irrelevant queries as well as irrelevant results.

The following section presents a technique to improve the quality of queries using LLMs by using a technique called ***query expansion***.

. . .

Query Expansion

The first type of query expansion that is going to be discussed is called:

Expansion with generated answers:

The way it works is you take your query and pass it through an LLM with you prompt to generate a hypothetical imagined answer to your query and then you concatenate your query with the imagined answer and use that as the new query which you pass to your retrieval system or vector database. Then you return your query results as normal.

```
from helper_utils import load_chroma, word_wrap, project_embeddings
from chromadb.utils.embedding_functions import SentenceTransformerEmbeddingFunct

embedding_function = SentenceTransformerEmbeddingFunction()

chroma_collection = load_chroma(filename='microsoft_annual_report_2022.pdf', col
chroma_collection.count()
```

```
import os
import openai
from openai import OpenAI

from dotenv import load_dotenv, find_dotenv
_ = load_dotenv(find_dotenv()) # read local .env file
openai.api_key = os.environ['OPENAI_API_KEY']

openai_client = OpenAI()

import umap

embeddings = chroma_collection.get(include=['embeddings'])['embeddings']
umap_transform = umap.UMAP(random_state=0, transform_seed=0).fit(embeddings)
projected_dataset_embeddings = project_embeddings(embeddings, umap_transform)
```

```
def augment_query_generated(query, model="gpt-3.5-turbo"):
    messages = [
        {
            "role": "system",
            "content": "You are a helpful expert financial research assistant. P
        },
        {"role": "user", "content": query}
    ]

    response = openai_client.chat.completions.create(
        model=model,
        messages=messages,
    )
    content = response.choices[0].message.content
    return content
```

```
original_query = "Was there significant turnover in the executive team?"
hypothetical_answer = augment_query_generated(original_query)

joint_query = f"{original_query} {hypothetical_answer}"
print(word_wrap(joint_query))
```

```
'''
OUTPUT:

Was there significant turnover in the executive team? "In the past
fiscal year, there was minimal turnover in the executive team with only
one change. The Chief Financial Officer retired and was succeeded by an
internal candidate, ensuring a smooth transition and continuity in
leadership."
'''
```

```
results = chroma_collection.query(query_texts=joint_query, n_results=5, include=
retrieved_documents = results['documents'][0]

for doc in retrieved_documents:
    print(word_wrap(doc))
    print('')
```

```
'''
OUTPUT:

87 changes in and disagreements with accountants on accounting and
financial disclosure not applicable. controls and procedures under the
supervision and with the participation of our management, including the
chief executive officer and chief financial officer, we have evaluated
the effectiveness of our disclosure controls and procedures as required
...
judgment and include complex considerations of the internal revenue
code, related regulations, tax case laws, and prior - year audit
settlements. given the complexity and the subjective nature of the
transfer pricing issues that remain
'''
```

```
retrieved_embeddings = results['embeddings'][0]
original_query_embedding = embedding_function([original_query])
augmented_query_embedding = embedding_function([joint_query])

projected_original_query_embedding = project_embeddings(original_query_embedding
projected_augmented_query_embedding = project_embeddings(augmented_query_embeddi
projected_retrieved_embeddings = project_embeddings(retrieved_embeddings, umap_t
```

```
import matplotlib.pyplot as plt

# Plot the projected query and retrieved documents in the embedding space
plt.figure()
plt.scatter(projected_dataset_embeddings[:, 0], projected_dataset_embeddings[:,
plt.scatter(projected_retrieved_embeddings[:, 0], projected_retrieved_embeddings
plt.scatter(projected_original_query_embedding[:, 0], projected_original_query_e
plt.scatter(projected_augmented_query_embedding[:, 0], projected_augmented_query

plt.gca().set_aspect('equal', 'datalim')
plt.title(f'{original_query}')
plt.axis('off')
```

According to the visualization above, the red cross is our original query, and the orange cross is our new query with the hypothetical answer. And we see that we get this good cluster of results. But most importantly, what I want to illustrate here is that using the hypothetical answer moves our query elsewhere in space, hopefully producing better results for us.

The second method of query expansion is:

Expansion with multiple queries

The way this works is by using the LLM to generate additional queries that might help answering the question. So what you do here is you take your original query, pass it to the LLM, ask the LLM to generate several new related queries to the same original query, and and then pass these new queries along with the original query to the vector database or your retrieval system. This gives the results for the original and the new queries, and then you pas all of those results to the LLM to complete the *RAG* loop.

```
def augment_multiple_query(query, model="gpt-3.5-turbo"):
    messages = [
        {
            "role": "system",
            "content": "You are a helpful expert financial research assistant. Y
            "Suggest up to five additional related questions to help them find t
            "Suggest only short questions without compound sentences. Suggest a
            "Make sure they are complete questions, and that they are related to
            "Output one question per line. Do not number the questions."
        },
        {"role": "user", "content": query}
    ]

    response = openai_client.chat.completions.create(
        model=model,
        messages=messages,
    )
    content = response.choices[0].message.content
    content = content.split("\n")
    return content
```

```
original_query = "What were the most important factors that contributed to incre
augmented_queries = augment_multiple_query(original_query)

for query in augmented_queries:
    print(query)
```

```
'''
OUTPUT:

- What were the net profits for the year?
- How did operating expenses change compared to the previous year?
- What were the key revenue drivers for each business segment?
- Were there any significant one-time gains or losses impacting the revenue?
- How did the revenue growth compare to industry benchmarks or competitors?
'''
```

```
queries = [original_query] + augmented_queries
results = chroma_collection.query(query_texts=queries, n_results=5, include=['do

retrieved_documents = results['documents']

# Deduplicate the retrieved documents
unique_documents = set()
for documents in retrieved_documents:
    for document in documents:
        unique_documents.add(document)

for i, documents in enumerate(retrieved_documents):
    print(f"Query: {queries[i]}")
    print('')
    print("Results:")
    for doc in documents:
        print(word_wrap(doc))
        print('')
    print('-'*100)
```

```
'''
OUTPUT:

Query: What were the most important factors that contributed to increases
in revenue?

Results:
engineering, gaming, and linkedin. • sales and marketing expenses
increased $ 1. 7 billion or 8 % driven by investments in commercial
sales and linkedin. sales and marketing included a favorable foreign
currency impact of 2 %. • general and administrative expenses increased
$ 793 million or 16 % driven by investments in corporate functions.
operating income increased $ 13. 5 billion or 19 % driven by growth
across each of our segments. current year net income and diluted eps
...
unfavorable foreign currency impact of 2 %. intelligent cloud revenue
increased $ 15. 2 billion or 25 %. • server products and cloud services
revenue increased $ 14. 7 billion or 28 % driven by azure and other
cloud services. azure and other cloud services revenue grew 45 % driven
by growth in our consumption - based

--------------------------------------------------------------------------------
'''
```

```
original_query_embedding = embedding_function([original_query])
augmented_query_embeddings = embedding_function(augmented_queries)

project_original_query = project_embeddings(original_query_embedding, umap_trans
project_augmented_queries = project_embeddings(augmented_query_embeddings, umap_
```

```
result_embeddings = results['embeddings']
result_embeddings = [item for sublist in result_embeddings for item in sublist]
projected_result_embeddings = project_embeddings(result_embeddings, umap_transfo
```

```
import matplotlib.pyplot as plt

plt.figure()
plt.scatter(projected_dataset_embeddings[:, 0], projected_dataset_embeddings[:,
plt.scatter(project_augmented_queries[:, 0], project_augmented_queries[:, 1], s=
plt.scatter(projected_result_embeddings[:, 0], projected_result_embeddings[:, 1]
plt.scatter(project_original_query[:, 0], project_original_query[:, 1], s=150, m

plt.gca().set_aspect('equal', 'datalim')
plt.title(f'{original_query}')
plt.axis('off')
```

As you see, using query expansion we're able to hit other related parts of the dataset that the single original query may not have reached. This gives us more of chance to find all of the related information, especially in the context of more complex queries, which require more and different types of information to answer. So in the projection above we see that the red cross is the original query, and the orange crosses are the augmented; the new queries generated for us by the LLM, and then once again, the green circles represent the results that we actually return by the retrieval system to the model. One way to think about this is that a single query turns into a single point in embedding space. And a single point in embedding space likely doesn't contain all of the information that you need to answer more complex query like this one. So using this form of query expansion, where we generate multiple related queries using an LLM gives us a better chance of capturing all of the related information.

The downside of this is of course now we have a lot more results than we had originally. And we're not sure if and which of these results are actually relevant to our query.

In the next section, using ***Cross-Encoder Re-ranking*** we have techniques that allows us to actually score the relevancy of all the returned results and use only the ones we feel match our original query.

. . .

Cross-Encoder Re-ranking

The *Cross-Encoder Re-ranking* is used to score the relevancy of the retrieved results for the sent query. Re-ranking is a way to reorder results and score them according to their relevancy to a particular query.

In re-ranking, after results have been retrieved for a particular query, you pass these results along with your query to a re-ranking model. This allows you to re-rank the output, so the most relevant results have the highest rank. In other words, your re-ranking the scores each of the results conditioned on the query, and those with the highest score are the most relevant. Then you can just select the top k re-ranked results as the most relevant to your particular query.

The following diagram illustrates each of the *Bi-Encoder* and *Cross-Encoder*.

```
from helper_utils import load_chroma, word_wrap, project_embeddings
from chromadb.utils.embedding_functions import SentenceTransformerEmbeddingFunct
import numpy as np

embedding_function = SentenceTransformerEmbeddingFunction()

chroma_collection = load_chroma(filename='microsoft_annual_report_2022.pdf', col
chroma_collection.count()
```

Re-ranking the long tail

```
query = "What has been the investment in research and development?"
results = chroma_collection.query(query_texts=query, n_results=10, include=['doc

retrieved_documents = results['documents'][0]

for document in results['documents'][0]:
    print(word_wrap(document))
    print('')
```

```
"""
OUTPUT:

• operating expenses increased $ 1. 5 billion or 14 % driven by
investments in gaming, search and news advertising, and windows
marketing. operating expenses research and development ( in millions,
except percentages ) 2022 2021 percentage change research and
development $ 24, 512 $ 20, 716 18 % as a percent of revenue 12 % 12 %
0ppt research and development expenses include payroll, employee
...
positive by 2030, zero waste by 2030, and to protect ecosystems by
developing a planetary computer. we also help our suppliers and
customers around the world use microsoft technology to reduce their own
carbon footprint."
"""
```

```
from sentence_transformers import CrossEncoder
cross_encoder = CrossEncoder('cross-encoder/ms-marco-MiniLM-L-6-v2')

pairs = [[query, doc] for doc in retrieved_documents]
scores = cross_encoder.predict(pairs)
print("Scores:")
for score in scores:
    print(score)
```

```
'''
OUTPUT:

Scores:
0.98693466
2.644579
-0.26802942
-10.73159
-7.7066045
-5.6469955
-4.297035
-10.933233
-7.0384283
-7.3246956
'''
```

You can see from the output above, that e.g., the second score of the result is of much higher score than the first one, and other scores down that are much higher than previous/ earlier ones.

```
print("New Ordering:")
for o in np.argsort(scores)[::-1]:
    print(o+1)
```

```
'''
OUTPUT:

New Ordering:
2
1
3
7
6
9
10
5
4
8
'''
```

According to the output above, if we were to take the first 5 re-ranked results, the final results would be much better, as the documents contain more relevant information.

Re-ranking with Query Expansion

Here, the Cross-Encoder re-ranking technique can be used to get all of the best results for the original query from the augmented expanded queries instead of just sending all of them to the LLM. The following shows how this can be done.

```
original_query = "What were the most important factors that contributed to incre
generated_queries = [
    "What were the major drivers of revenue growth?",
    "Were there any new product launches that contributed to the increase in rev
    "Did any changes in pricing or promotions impact the revenue growth?",
    "What were the key market trends that facilitated the increase in revenue?",
    "Did any acquisitions or partnerships contribute to the revenue growth?"
]
```

```
queries = [original_query] + generated_queries

results = chroma_collection.query(query_texts=queries, n_results=10, include=['d
retrieved_documents = results['documents']

# Deduplicate the retrieved documents
unique_documents = set()
for documents in retrieved_documents:
    for document in documents:
        unique_documents.add(document)

unique_documents = list(unique_documents)

pairs = []
for doc in unique_documents:
    pairs.append([original_query, doc])

scores = cross_encoder.predict(pairs)

print("Scores:")
for score in scores:
    print(score)
```

```
'''
OUTPUT:

Scores:
-10.042844
-8.505108
-10.00014
-4.651892
-9.80788
-7.754099
-9.918428
-4.8184834
-7.9171767
-9.768024
-1.136996
-10.0839405
-9.357721
-11.0792675
-6.90209
-10.148884
-4.3417687
-5.141832
-3.7948632
-7.4906564
-5.274752
-3.7681527
-10.711212
'''
```

```
print("New Ordering:")
for o in np.argsort(scores)[::-1]:
    print(o)
```

```
'''
OUTPUT:

New Ordering:
10
21
18
16
3
7
17
20
14
19
5
8
1
12
9
4
6
2
0
11
15
22
13
'''
```

Now if we take the top *k* results, we guarantee that they contain the most relevant information to be passed into the LLM.

. . .

Query Adapters

Query adapters are a way to directly alter or augment the query embedding itself using user feedback or other types of data to get better query results.

In effect, another additional stage in the retrieval system is inserted, called an embedding adapter, which happens after the embedding model, but before we retrieve the most relevant results. The embedding adapter is trained using the user feedback of the relevancy of our retrieved results for our set of queries.

```
from helper_utils import load_chroma, word_wrap, project_embeddings
from chromadb.utils.embedding_functions import SentenceTransformerEmbeddingFunct
import numpy as np
import umap
from tqdm import tqdm

import torch


embedding_function = SentenceTransformerEmbeddingFunction()

chroma_collection = load_chroma(filename='microsoft_annual_report_2022.pdf', col
chroma_collection.count()

embeddings = chroma_collection.get(include=['embeddings'])['embeddings']
umap_transform = umap.UMAP(random_state=0, transform_seed=0).fit(embeddings)
projected_dataset_embeddings = project_embeddings(embeddings, umap_transform)
```

```
import os
import openai
from openai import OpenAI

from dotenv import load_dotenv, find_dotenv
_ = load_dotenv(find_dotenv()) # read local .env file
openai.api_key = os.environ['OPENAI_API_KEY']

openai_client = OpenAI()
```

Creating a Dataset

The first thing required for this approach is a dataset. However, we don’t have one ready because we haven’t really had users using our *RAG* application. But we can use a model to generate a dataset for us. In fact, this is about creating the right prompt. Hence, *GPT* is gonna be used again. And essentially, we’re prompting the model as in previous sections, in addition to prompting it to generate the dataset.

```
def generate_queries(model="gpt-3.5-turbo"):
    messages = [
        {
            "role": "system",
            "content": "You are a helpful expert financial research assistant. Y
            "Suggest 10 to 15 short questions that are important to ask when ana
            "Do not output any compound questions (questions with multiple sente
            "Output each question on a separate line divided by a newline."
        },
    ]

    response = openai_client.chat.completions.create(
        model=model,
        messages=messages,
    )
    content = response.choices[0].message.content
    content = content.split("\n")
    return content

generated_queries = generate_queries()
for query in generated_queries:
    print(query)
```

```
'''
OUTPUT:

1. What is the overall financial performance of the company in the past year?
2. How have the revenues and expenses of the company changed year-over-year?
3. What is the company's net income and how does it compare to previous years?
4. What are the company's key sources of revenue and how have they evolved over
5. How much debt does the company have and how does it compare to its equity?
6. What is the company's cash flow from operating activities and how does it com
7. What are the company's major expenses and how have they changed over the past
8. What is the company's current ratio and quick ratio, and what do they indicat
9. How have the company's assets and liabilities changed over the past year?
10. What is the company's return on investment and how does it compare to indust
11. Does the company pay dividends and how have they changed over time?
12. What is the company's strategy for future growth and how does it plan to fin
13. What are the company's major risks and challenges going forward?
14. How does the company's financial performance compare to its competitors in t
15. What accounting policies and estimates does the company use and how do they

'''
```

```
results = chroma_collection.query(query_texts=generated_queries, n_results=10, i
retrieved_documents = results['documents']
```

The following is an evaluation script, we’re gonna ask the financial expert to tell us whether a given statement is relevant to a given query. And we’re gonna ask it to output only yes or no.

```
def evaluate_results(query, statement, model="gpt-3.5-turbo"):
    messages = [
    {
        "role": "system",
        "content": "You are a helpful expert financial research assistant. You h
        "For the given query, evaluate whether the following satement is relevan
        "Output only 'yes' or 'no'."
    },
    {
        "role": "user",
        "content": f"Query: {query}, Statement: {statement}"
    }
    ]

    response = openai_client.chat.completions.create(
        model=model,
        messages=messages,
        max_tokens=1
    )
    content = response.choices[0].message.content
    if content == "yes":
        return 1
    return -1
```

```
retrieved_embeddings = results['embeddings']
query_embeddings = embedding_function(generated_queries)
```

```
adapter_query_embeddings = []
adapter_doc_embeddings = []
adapter_labels = []

for q, query in enumerate(tqdm(generated_queries)):
    for d, document in enumerate(retrieved_documents[q]):
        adapter_query_embeddings.append(query_embeddings[q])
        adapter_doc_embeddings.append(retrieved_embeddings[q][d])
        adapter_labels.append(evaluate_results(query, document))

print(len(adapter_labels))
# OUTPUT: 150
```

```
adapter_query_embeddings = torch.Tensor(np.array(adapter_query_embeddings))
adapter_doc_embeddings = torch.Tensor(np.array(adapter_doc_embeddings))
adapter_labels = torch.Tensor(np.expand_dims(np.array(adapter_labels),1))

dataset = torch.utils.data.TensorDataset(adapter_query_embeddings, adapter_doc_e
```

Setting up the model

The following is a step-by-step approach to set up and train the model.

```
def model(query_embedding, document_embedding, adaptor_matrix):
    updated_query_embedding = torch.matmul(adaptor_matrix, query_embedding)
    return torch.cosine_similarity(updated_query_embedding, document_embedding,

def mse_loss(query_embedding, document_embedding, adaptor_matrix, label):
    return torch.nn.MSELoss()(model(query_embedding, document_embedding, adaptor

# Initialize the adaptor matrix
mat_size = len(adapter_query_embeddings[0])
adapter_matrix = torch.randn(mat_size, mat_size, requires_grad=True)

min_loss = float('inf')
best_matrix = None

for epoch in tqdm(range(100)):
    for query_embedding, document_embedding, label in dataset:
        loss = mse_loss(query_embedding, document_embedding, adapter_matrix, lab

        if loss < min_loss:
            min_loss = loss
            best_matrix = adapter_matrix.clone().detach().numpy()

        loss.backward()
        with torch.no_grad():
            adapter_matrix -= 0.01 * adapter_matrix.grad
            adapter_matrix.grad.zero_()
```

Now one thing we’d like to take a look at is how the adapter matrix influences our query vector. To do that, we can construct a test vector consisting of all ones, and we can multiply that test vector by our best matrix. And what this will tell us is which dimensions of our vectors get scaled by what amount. One can think of an embedding adapter as stretching space and squeezing space for the dimensions which are most relevant to the particular queries that we have while reducing the dimensions that are not relevant to our query. You’ll also notice that it can reverse dimensions. The following plot shows how it looks like.

```
print(f"Best loss: {min_loss.detach().numpy()}")
# OUTPUT: Best loss: 0.581408679485321
```

```
test_vector = torch.ones((mat_size,1))
scaled_vector = np.matmul(best_matrix, test_vector).numpy()

import matplotlib.pyplot as plt
plt.bar(range(len(scaled_vector)), scaled_vector.flatten())
plt.show()
```

Here you can see how each dimension of our test vector, which consists only of ones, has been stretched and squeezed. Some have been elongated a lot, while others have been made to be almost zero. And so what this means is our embedding adapter has basically decided that these dimensions (to the left) are more relevant where those (to the right) are less relevant.

The following illustrate how this affects the queries.

```
query_embeddings = embedding_function(generated_queries)
adapted_query_embeddings = np.matmul(best_matrix, np.array(query_embeddings).T).

projected_query_embeddings = project_embeddings(query_embeddings, umap_transform
projected_adapted_query_embeddings = project_embeddings(adapted_query_embeddings
```

```
# Plot the projected query and retrieved documents in the embedding space
plt.figure()
plt.scatter(projected_dataset_embeddings[:, 0], projected_dataset_embeddings[:,
plt.scatter(projected_query_embeddings[:, 0], projected_query_embeddings[:, 1],
plt.scatter(projected_adapted_query_embeddings[:, 0], projected_adapted_query_em

plt.gca().set_aspect('equal', 'datalim')
plt.title("Adapted Queries")
plt.axis('off')
plt.legend()
```

As you can see, our original queries were quite scattered around, but our new queries concentrate in a certain part of the dataset which is more relevant to our queries. You can see how the red queries have been adapted through the embedding adapter to transform them into the green queries, to push them into a particular part of the space. So as you can see, an embedding adapter is a simple but powerful technique for customizing query embeddings to your specific application. In order to make this work, you need to collect a dataset, either a synthetic one like the one we’ve generated before, or else one that’s based on user data. User data usually works best because it actually means that people are using your retrieval system for their specific tasks. Again, because this approach involves prompting and because it involves the use of an LLM, it’s worth experimenting with the prompts and with different initializations of the adapters.

. . .

Conclusion

Embedding space retrieval is still an active area of research, and there's a lot of other techniques that one should be aware of. For instance, you can fine-tune the embedding model directly using the same type of data as used in a previous section (embedding adapters). Additionally, recently there's been some really good results published in fine-tuning the LLM itself to expect retrieved results and reason about them (check references and resources). Additionally, you could experiment with a more complicated embedding adapter model using a full-blown neural network or even a transformer layer. Similarly, you can use a more complex relevance modeling model rather that just using the Cross-Encoder as described earlier in this article. Furthermore, an often overlooked piece is that the quality of retrieved results often depends on the way that your data is chunked before it's stored in the retrieval system itself. There's a lot of experiments going on right now about using deep models including transformers for optimal and intelligent chunking.

. . .

References and Resources

- OpenAI short course: Advanced Retrieval for AI with Chroma
- Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks
- Expansion with generated answers
- RA-DIT: Retrieval-Augmented Dual Instruction Tuning
- InstructRetro: Instruction Tuning post Retrieval-Augmented Pretraining

Rag Llms Cross Encoder Query Expansion Chroma

41

Written by Sulaiman Shamasna

244 followers · 122 following

Follow

An experienced Data Scientist and Machine Learning Engineer with main focus on LLMs & MLOps. In addition to a deep background in Philosophy, Physics, and Maths.